

```
--- djangoapi\core\myLib\managePermissions.py ---
```

```
'''
```

```
Created on 12 dic 2023
```

```
@author: joamona
```

```
'''
```

```
#Para añadir los permisos se crea un modelo como este, y se ponen los nombres de las  
url, o vistas que se quiere chequear
```

```
# en el atributo permission.
```

```
from typing import Union, List
```

```
from django.contrib.auth.models import Group, Permission
```

```
from django.contrib.auth.models import User
```

```
from django.http import JsonResponse
```

```
from djangoapi import settings
```

```
from django.db import models
```

```
from django.contrib.auth.models import Permission, ContentType
```

```
from core.myLib.generalModule import stringListToString
```

```
'''
```

```
    Ejecutar lo siguiente para crear los permisos:
```

```
        python manage.py makemigrations
```

```
        python manage.py migrate
```

```
    Si te equivocas, puedes borrar los permisos erróneos de la tabla manualmente.
```

```
        public.auth_permission
```

```
'''
```

```
class CreatePermissions(models.Model):
```

```
    class Meta:
```

```
        managed = False # No database table creation, or deletion operations will be  
performed for this model.
```

```
        default_permissions = () # disable "add", "change", "delete" and "view"
```

```
default permissions
```

```
    permissions = (
```

```
        ('building_insert_2', 'building_insert_2'),
```

```
        ('BuildingSelect', 'BuildingSelect')
```

```
    )
```

```
def getUserGroups(user: User):
```

```
    '''
```

```
    Gets a list of the names of the user groups (list of strings). The user is an object  
of the
```

```
        django.contrib.auth.models.User class
```

```
    '''
```

```
    l = user.groups.values_list('name', flat = True) # QuerySet Object
```

```
    return list(l)
```

```
def getUserGroupsToString(user: User):
```

```
    '''
```

```

    Gets a single string with the user groups that the user belongs, separated by
commas.
    """
    return stringListToString(getUserGroups(user))

def getUserGroupsToString_fromUsername(username):
    user=User.objects.get(username=username)
    return getUserGroupsToString(user)

def getUserGroups_fromUsername(username):
    """
    Gets a lists with the user groups that the user belongs. The username is the
username,
usually an email
    """
    user=User.objects.get(username=username)
    return getUserGroups(user)

def addOrGetGroup(groupName: str):
    """
    Añade o recupera un grupo. Si no existe lo crea, si existe lo recupera. Devuelve el
grupo creado o recuperado
    Ejemplo:
        addOrGetGroup('prueba_grupo')
    """
    newGroup, existingGroup = Group.objects.get_or_create(name=groupName)
    if not(newGroup is None):
        return newGroup
    else:
        return existingGroup

def createGroups(groupNamesList: list):
    """
    Añade los grupos de la lista si no existen. Si no existe lo crea, si existe lo
recupera. Devuelve una lista con
los grupos creados o recuperados.
    Ejemplo:
        createGroups(['prueba_grupo', 'grupo1', 'grupo3'])
    """
    l=[]
    for g in groupNamesList:
        l.append(addOrGetGroup(groupName=g))
    return l

def addPermissionToGroup(groupObject, permissionName: str):
    print(permissionName)
    permission = list(Permission.objects.filter(codename=permissionName))[0]
    groupObject.permissions.add(permission)

def addPermissionToGroupName(groupName: str, permissionName: str):
    groupObject=list(Group.objects.filter(name=groupName))[0]
    addPermissionToGroup(groupObject, permissionName)

```

```

def addPermissionsToGroupName(groupName, permissionNamesList):
    """
    Añade los permisos a un grupo. Da error si el grupo, o el permiso no existe
    Ejemplo:

    permissionNamesList=['building_insert','building_delete','building_delete_all','building_update']

    general.addPermissionsToGroup(groupName='prueba_grupo',
    permissionNamesList=permissionList)
    """
    for permissionName in permissionNamesList:
        addPermissionToGroupName(groupName, permissionName)

def removePermissionFromGroup(groupObject, permissionName: str):
    permission = list(Permission.objects.filter(codename=permissionName))[0]
    groupObject.permissions.remove(permission)

def removePermissionFromGroupName(groupName, permissionName: str):
    groupObject=list(Group.objects.filter(name=groupName))[0]
    removePermissionFromGroup(groupObject, permissionName)

def removePermissionsFromGroupName(groupName, permissionNamesList: list):
    """
    Borra los permisos de un grupo. No da error si el grupo no tiene el permiso.
    Sí da error si el nombre del permiso no existe.
    Ejemplo:

    permissionNamesList=['building_insert','building_delete','building_delete_all','building_update']

    general.removePermissionsFromGroupName('prueba_grupo', permissionNamesList)
    """
    for permissionName in permissionNamesList:
        removePermissionFromGroupName(groupName, permissionName)

def listPermissions():
    permissions = Permission.objects.all()
    for per in permissions:
        print(per)
        print(per.__dict__)

def check():
    permission = list(Permission.objects.filter(codename='building_delete'))[0]
    print(permission.__dict__)

#35.21 Controlar acceso a las vistas mediante la url, o el nombre de la vista
class CheckAccessToView():
    """
    Comprueba si el usuario almacenado en el request está en algún grupo que tenga un
    permiso que
    se llame como el nombre de la vista a la que está accediendo, anteponiendo el nombre
    de la app: appdesweb.BuildingSelect.
    """

```

Para añadir los permisos se crea un modelo como este, y se ponen los nombres de las url, o vistas que se quiere chequear en el atributo permission.

```
class CreatePermissions(models.Model):
    class Meta:
        managed = False # No database table creation, or deletion operations will be
        performed for this model.
        default_permissions = () # disable "add", "change", "delete" and "view"
default permissions
        permissions = (
            ('building_insert_2', 'building_insert_2'),
            ('BuildingSelect', 'BuildingSelect')
        )
```

Ejecutar lo siguiente para crear los permisos:

```
python manage.py makemigrations
python manage.py migrate
```

Si te equivocas, puedes borrar los permisos erróneos de la tabla manualmente.

```
public.auth_permission
```

```
--
```

Esta clase debe usarse cuando la url tiene parámetros, por ejemplo la siguiente url /building_select/125/

no puede usarse con la clase CheckAccessToUrl, entonces se puede usar CheckAccessToView. Para todas las operaciones post, y get que no tengan parámetros debe usarse la clase CheckAccessToUrl.

Siempre se usa de la misma manera. Copiar y pegar en la vista. Devolverá un json en el caso de que no tenga permiso.

Si se está en modo debug el json da más detalles.

Ejemplo de uso:

```
CheckAccessToView.__init__(self, request,self.__class__.__name__)
if self.no_perm_to_use_view:
    return self.no_perm_json_response
"""

viewClassName=None
no_perm_to_use_view=True
no_perm_json_response=JsonResponse({'ok':False, 'message':'You can not access to
this operation', 'data':None})
def __init__(self, request, viewClassName):
    APP_LABEL='SOLUCIONAR LO DEL NOMBRE DE LA APP'
    self.viewClassName = viewClassName
    self.no_perm_to_use_view=not(request.user.has_perm(APP_LABEL + '.' +
viewClassName))#true if not perm
    if self.no_perm_to_use_view:
        if settings.DEBUG:
            l_groups = list(request.user.groups.values_list('name',flat = True)) #
QuerySet Object
            self.no_perm_json_response = JsonResponse({'ok':False,
```

```

        'message':'You can not access
to the view ' + viewClassName,

'data':[{'username':request.user.username,
        'groups': l_groups
        }]
    })

```

```

class CheckAccessToUrl():
    """
    Se puede usar como clase base para vistas que devuelven un json. La vista devolverá
    un JsonResponse de falta de permisos, aunque el usuario esté autenticado. El json es
    el de siempre:

```

```

        {'ok':False, 'message':'You can not access to this operation', 'data':None}

```

Si debug es true da más detalles en la respuesta.

Usa la url accedida para comprobar si el usuario tiene un permiso con el mismo nombre.

Para acceder a la url: '/building_insert/' el usuario debe pertenecer a un grupo que tenga el permiso 'building_insert'.

Comprueba si el usuario almacenado en el request está en algún grupo que tenga un permiso que

se llame como la url a la que está accediendo, anteponiendo el nombre de la app: appdesweb.building_insert.

La url debe definirse en el fichero urls.py de esta forma

```

    path('building_insert/',...)

```

o esta forma

```

    path('building_select/222/',

```

Si no es así, se puede usar CheckAccessToView, que usa el nombre de la vista para saber si hay permisos.

Para añadir los permisos se crea un modelo como este, y se ponen los nombres de las url, o vistas que se quiere chequear en el atributo permission.

```

class CreatePermissions(models.Model):
    class Meta:
        managed = False # No database table creation, or deletion operations will be
        performed for this model.
        default_permissions = () # disable "add", "change", "delete" and "view"
default permissions
        permissions = (
            ('building_insert_2', 'building_insert_2'),
            ('BuildingSelect', 'BuildingSelect')
        )

```

Ejecutar lo siguiente para crear los permisos:

```

python manage.py makemigrations

```

```
python manage.py migrate
```

Si te equivocas, puedes borrar los permisos erróneos de la tabla manualmente.

```
public.auth_permission
```

--

Siempre se usa de la misma manera. Copiar y pegar en la vista. Devolverá un json en el caso de que no tenga permiso.

Si se está en modo debug el json da más detalles.

Ejemplo de uso:

Heredar de esta clase:

```
class BuildingSelect(LoginRequiredMixin,View,CheckAccessToUrl):

    CheckAccessToUrl.__init__(self, request)
    if self.no_perm_to_use_view:
        return self.no_perm_json_response
    """
    no_perm_to_use_view=True
    no_perm_json_response=JsonResponse({'ok':False, 'message':'You can not access to
this operation', 'data':None})
    def __init__(self, request):
        APP_LABEL='SOLUCIONAR LO DEL NOMBRE DE LA APP'
        url_tried = request.path_info.split('/')[1]
        self.no_perm_to_use_view=not(request.user.has_perm(APP_LABEL + '.' +
url_tried))#true if not perm
        if self.no_perm_to_use_view:
            if settings.DEBUG:
                l_groups = list(request.user.groups.values_list('name',flat = True)) #
QuerySet Object
                self.no_perm_json_response = JsonResponse({'ok':False,
                                                            'message':'You can not access
to the url ' + url_tried,

'data':[{'username':request.user.username,
                                                'groups': l_groups
                                                }]}
            })

    def addUserToGroup(username, groupname):
        g = Group.objects.get(name=groupname)
        u = User.objects.get(username=username)
        g.user_set.add(u)

    def create_permissions(permissions_list):
        """
        Crea permisos en la tabla auth_permission.
        Los permisos quedan asociados a la app auth. Para saber si tiene el permiso
        el código es auth.codename
```

Args:

permissions_list (list): Lista de diccionarios, cada uno con los campos 'name' y 'codename'.

Ejemplo:

```
permissions = [
    {'name': 'Can view report', 'codename': 'can_view_report'},
    {'name': 'Can edit settings', 'codename': 'can_edit_settings'},
]
create_permissions(permissions)
```

```
managePermissions.addPermissionsToGroupName('gestor_municipal', ['can_view_report',
'can_edit_settings'])
```

```
managePermissions.addUserToGroup('j.cami2010@hotmail.com', 'gestor_municipal')
```

```
joa=User.objects.get(username='joamona@cgf.upv.es')
```

```
camí=User.objects.get(username='j.cami2010@hotmail.com')
```

```
print(joa.has_perm('auth.can_view_report')) -> False si no es admin
```

```
print(camí.has_perm('auth.can_view_report')) -> True
```

```
"""
```

```
# Obtener el ContentType genérico para los permisos globales
```

```
content_type, created = ContentType.objects.get_or_create(
```

```
    app_label='auth', model='permission'
```

```
)
```

```
for permission_data in permissions_list:
```

```
    name = permission_data.get('name')
```

```
    codename = permission_data.get('codename')
```

```
    if not name or not codename:
```

```
        print(f"Saltando permiso inválido: {permission_data}")
```

```
        continue
```

```
    # Crear o actualizar el permiso
```

```
    permission, created = Permission.objects.get_or_create(
```

```
        codename=codename,
```

```
        content_type=content_type,
```

```
        defaults={'name': name}
```

```
)
```

```
    if created:
```

```
        print(f"Permiso creado: {name} ({codename})")
```

```
    else:
```

```
        print(f"El permiso ya existe: {name} ({codename})")
```

```
def isAdministrator(request):
```

```
    user=request.user
```

```
    return user.groups.filter(name='admin').exists()
```

```
def getUserGroupsAsDict(user_username_or_id: Union[User, str, int]):
```

```
    try:
```

```

    # Detectar si el identificador es un entero (ID) o una cadena (username)
    if isinstance(user_username_or_id, int):
        user = User.objects.get(id=user_username_or_id)
    elif isinstance(user_username_or_id, str):
        user = User.objects.get(username=user_username_or_id)
    elif isinstance(user_username_or_id, User):
        user = user_username_or_id
    else:
        raise ValueError("El identificador debe ser un ID (int) o un nombre de
usuario (str).")

    # Obtener los grupos del usuario
    groups = user.groups.values('id', 'name')
    return list(groups)

except User.DoesNotExist:
    return []
except ValueError as e:
    return {"error": str(e)}

def user_belong_to_group(user:User, groupName:str) -> bool:
    return user.groups.filter(name=groupName).exists()

def user_belong_to_any_group_in_list(user:User, listOfNamesOfGroupsToCheck: list)->bool:
    l=getUserGroups(user)
    for groupToCkeck in listOfNamesOfGroupsToCheck:
        if groupToCkeck in l:
            return True
    return False

def user_belong_to_group_fromUsername(username:str, groupName:str) -> bool:
    user=User.objects.get(username=username)
    return user.groups.filter(name=groupName).exists()

def getGroupMembers(group_name:str)->list['str']:
    """
    Returns a list with the usernames of the users that belong to the group.
    """
    users=User.objects.filter(groups__name=group_name)
    l=[]
    for u in users:
        l.append(u.username)
    return l

```